

MACHINE REPRESENTATION OF PROGRAMS

Computers execute machine code

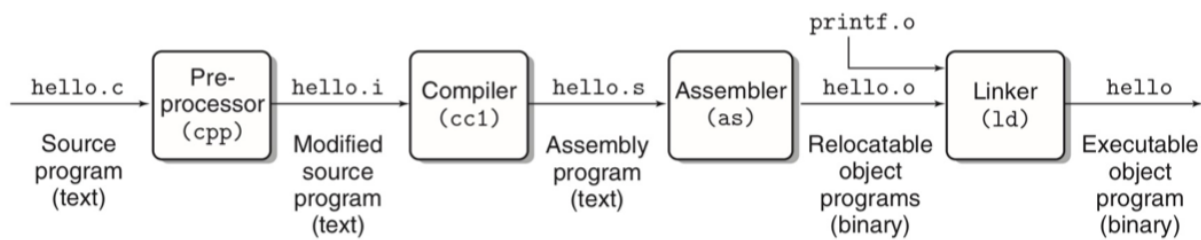
↳ sequence of bytes encoding low level operations



A compiler generates machine code by-

- Rules of the programming language
- Instruction set of the target machine
- Conventions of the Operating System

→ The Compilation Process



Compiler Optimizations can rearrange execution order, eliminate unwanted computations, replace slow operations and change recursive computation to iterative ones.

Moore's Law - The number of transistors per chip will double every year.

↳ Moore's Law has sadly ended. I believe Physics caught up.

The format and behavior of machine level programs is defined by the instruction set architecture. Most ISAs describe the behavior of the program as if each instruction is executed in sequence, with one instruction completing before the next one begins.

High level languages provide a model in which objects of different data types can be declared and allocated in memory, machine code views the memory as simply a large byte-addressable array.

Program memory is addressed using virtual addresses. The OS manages the virtual address space, translating virtual addresses into the physical addresses.

To view the assembly code generated by the C compiler-

> gcc -Og -S hello.c

-Og corresponds to the level of compiler optimization

Disassemblers allow us to inspect contents of the machine level code. They generate a format similar to assembly code from machine code.

x86-64 instructions can range in length from 1 to 15 bytes. Commonly used instructions and those with fewer operands require a smaller number of bytes.

Most assembly code instructions generated by GCC have a single character suffix denoting the size of the operand. Floating point code involves an entirely different set of instructions and registers.

An x86-64 CPU contains a set of 16 general purpose registers storing 64-bit values. These registers are used to store integer data as well as pointers -

%rax → Return Value

%rbx → Caller Saved

%rcx → 4th Argument

%rdx → 3rd Argument

%rsi → 2nd Argument

%rdi → 1st Argument

%rbp → Caller Saved

%rsp → Stack Pointer

%r8 → 5th Argument

%r9 → 6th Argument

%r10 → Caller Saved

%r11 → Caller Saved

%r12 → Caller Saved

%r13 → Caller Saved

%r14 → Caller Saved

%r15 → Caller Saved

→ What happens if we have more than 6 arguments?

• The excess arguments are placed on the stack. Note that accessing the stack is slower than registers.

Assembly Instructions - Instruction Operand [Operand]

↳ Operands are generally source & destination locations

constants, registers, memory ←

↳ registers & memory

→ Data Movement Instructions -

Mov S, D → This instruction copies data from a source location to a destination location without any transformation.

↳ x86 restricts both operands from referencing memory locations. This is because CPU buses are designed to only make one memory reference at a time. Copying a value from one memory location to another requires two instructions: the first to load the source value into a register and the second to write this register to the destination.

MOVZ S, R → Copies a smaller value to a larger destination.

↳ Fills the remaining bytes of the destination with zeroes.

MOVS S, R → Copies a smaller value to a larger destination.

↳ Fills the remaining bytes of the destination with sign extension (replicating copies of the MSB of the source operand).

NOTE: For most Mov instructions, only the specific register bytes mentioned by the operand are updated. However, when we have a 32 bit value to a 64 bit destination, the upper 32 bits are zeroed out.

NOTE: Pointers in C are simply addresses. Dereferencing a pointer involves copying that pointer into a register, and then using this register in a memory access.

NOTE: Local variables are stored in registers as opposed to memory. Register access is faster than memory access.

